

LITERATURE SURVEY

This work is dedicated to the design and implementation of an activation function unit based on the sigmoid function using Verilog on Vivado. An activation function unit is a fundamental building block used in digital neural network hardware to introduce nonlinearity and enable efficient learning and inference. The sigmoid function, in particular, is widely used because of its smooth, bounded, and monotonic characteristics, which map input values into a normalized output range between zero and one.

In this design, the input data is processed through a selective sign converter and domain decoder to determine the appropriate operating region of the sigmoid function. Based on the decoded domain, precomputed coefficients and modified input values are selected using multiplexers. These values are then combined using arithmetic units such as adders to generate the final output while handling saturation and most-significant-bit conditions to ensure numerical stability. The overall architecture is optimized for hardware efficiency and suitability for fixed-point implementation.

The proposed activation function unit is described at the register transfer level (RTL) using the hardware description language Verilog. The design is implemented, synthesized, and simulated using Xilinx Vivado (22.2). The block diagram of the activation function unit is shown in the figure, illustrating the modular structure and data flow within the system. This implementation demonstrates a practical and efficient hardware realization of the sigmoid activation function suitable for neural network accelerators and other signal processing applications.

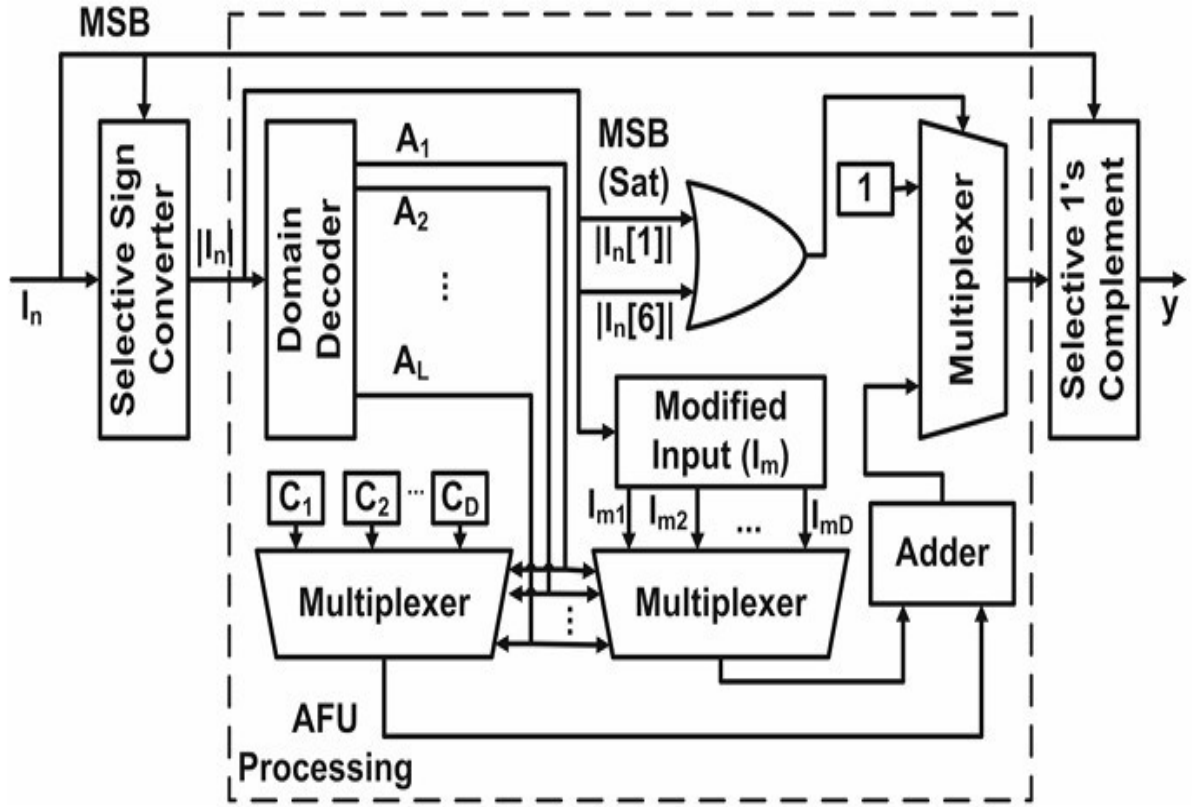
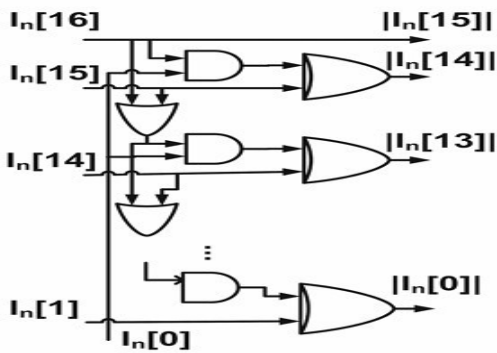
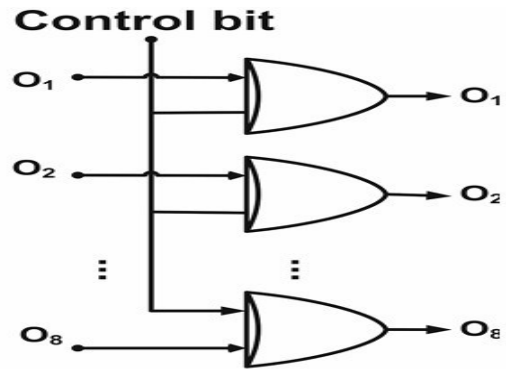


Fig. 6 Architecture of the proposed activation function unit



(a) Architecture of selective sign converter circuit



(b) Architecture of selective 1's complement circuit

INTRODUCTION

Various neural network operations used in day-to-day signal processing and AI applications can be represented using an AFU (Activation Function Unit) such as the sigmoid function.

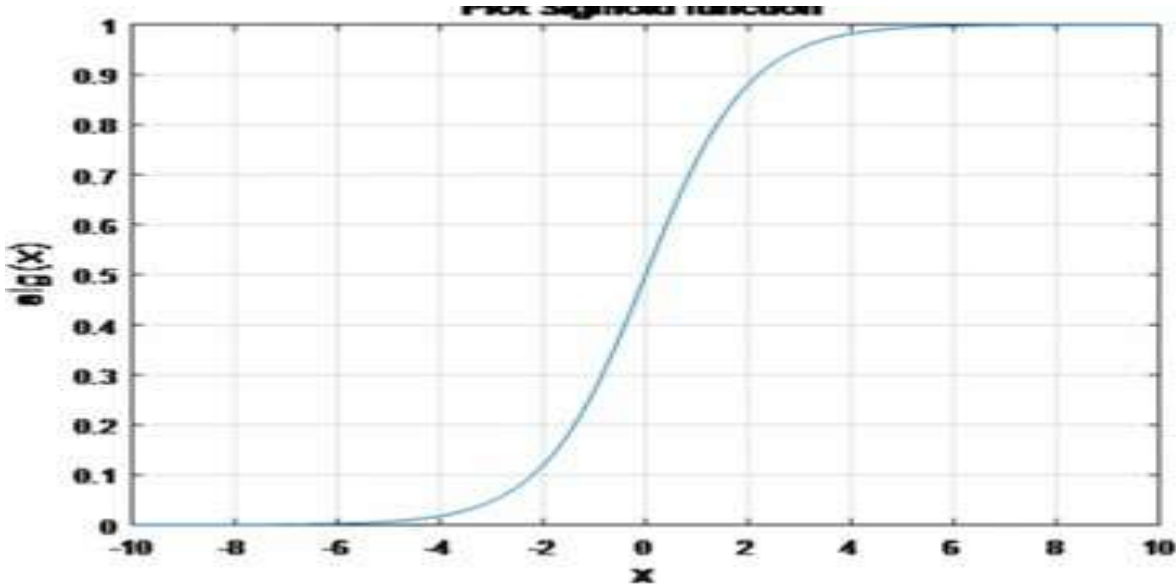
- An AFU is a nonlinear computational block that maps input data to a bounded output range and plays a crucial role in introducing nonlinearity in neural networks.
- The AFU operates by processing the input through different functional regions based on its magnitude and sign, and produces the corresponding activation output using predefined logic.
- An AFU is defined by its input format, functional domains, approximation method, and the control logic that selects the appropriate computation path.
- Here, the sigmoid activation function is divided into multiple operating regions such as saturation, linear approximation, and transition regions to simplify hardware realization.
- Now we will observe the operation of the AFU by writing a Verilog testbench, which verifies the functional correctness of the activation unit for different input conditions.

Organization of Proposed Activation Function Unit

This section presents the efficient organization of AFU, we have considered that AFU of perceptron is a ‘S’-shaped, sigmoid function. Figure2 depicts the plot of a sigmoidal AFU, i.e., $f(.)$, for input In , $f(.)$ can be defined As

$$f(In) = \left(\frac{1}{1 + e^{-In}} \right) \dots\dots\dots(4)$$

The sigmoid function defined by the above equation has a point symmetry about the point (0, 0.5), and the value of the function varies from 0 to 1 as the input In varies from $-\infty$ to $+\infty$. The observation of the plot presented in Fig.2 relates the three basic properties of this Sigmoid function as follows:



Property 1 Value of the sigmoid function is obtained as 0.5, if input In is equal to the 0.

$$f(In) = 0.5 \text{ at } In = 0. \quad (5)$$

Property 2 The sum of the sigmoid function values for a particular input In and its mirrored input $-In$ is unity, i.e.,

$$f(In) + f(-In) = 1, \quad (6)$$

or

$$f(-In) = 1 - f(In). \quad (7)$$

Property 3 For a permissible error limit of 0.02, sigmoid function attains saturation value as input $|In| \geq 8$.

$$f(I_n) = 1 \text{ for } I_n \geq 8, \quad (8)$$

and

$$f(I_n) = 0 \text{ for } I_n \leq -8, \quad (9)$$

organization of AFU can be depicted by a simple block diagram shown in 3. To evaluate both positive and negative values of I_n , we use a selective sign converter (SCC) that always provides the $|I_n|$ for the computation of activation function. The circuit diagram of SCC is shown in Fig.4a. The output of SCC is given to the AFU computation block shown in Fig.3. After the computation of activation function for $|I_n|$, the values are passed through the selective 1's complementor circuit shown in Fig.4b which converts it into 1's complement if the initial input I_n was negative.

Table 1 Approximation of the sigmoid function in the range of 0 to 8

Region	Equivalent function
$0 \leq I_n \leq 1$	$I_n/4 + 0.5$
$1 < I_n \leq 2.375$	$I_n/8 + 0.625$
$2.375 < I_n \leq 5$	$I_n/32 + 0.84375$
$5 < I_n < 8$	$I_n/64 + 0.9875$

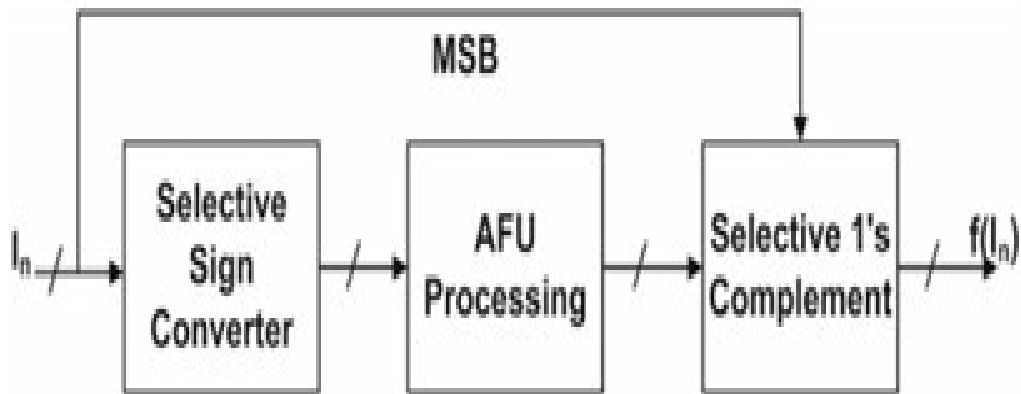


Fig. 3 Block diagram of AFU organization

Verilog Code

```
`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 30.08.2025 02:16:12
// Design Name:
// Module Name: aafu
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
/////////////////////////////////////////////////////////////////

//module aafu(
    // );

    module sgn(i0,i16,i15,f,g);
    input i0,i16,i15;
    output f,g;
    wire t1;
    assign t1=i0&i16;
    assign g=i15|i16;
    assign f=i15^t1;
    endmodule

module SIGN_conv(integerr,sgn,fraction,modIn );
    input sgn;
    input [1:9] integerr;
    input [10:16] fraction;
```

```

output [0:15] modIn;
wire [2:15] t;
wire w;
assign modIn[15]=fraction[16];

sgn g16(sgn,fraction[16],fraction[15],modIn[14],t[15]);
sgn g15(sgn,t[15],fraction[14],modIn[13],t[14]);
sgn g14(sgn,t[14],fraction[13],modIn[12],t[13]);
sgn g13(sgn,t[13],fraction[12],modIn[11],t[12]);
sgn g12(sgn,t[12],fraction[11],modIn[10],t[11]);
sgn g11(sgn,t[11],fraction[10],modIn[9],t[10]);
sgn g10(sgn,t[10],integerr[9],modIn[8],t[9]);
sgn g9(sgn,t[9],integerr[8],modIn[7],t[8]);
sgn g8(sgn,t[8],integerr[7],modIn[6],t[7]);
sgn g7(sgn,t[7],integerr[6],modIn[5],t[6]);
sgn g6(sgn,t[6],integerr[5],modIn[4],t[5]);
sgn g5(sgn,t[5],integerr[4],modIn[3],t[4]);
sgn g4(sgn,t[4],integerr[3],modIn[2],t[3]);
sgn g3(sgn,t[3],integerr[2],modIn[1],t[2]);
assign w=t[3]&sgn;
assign modIn[0]=integerr[1]^w; endmodule

```

```

module DOMAIN_Decoder(in,y

```

```

);
input [7:16] in;
output reg [1:0] y;
always @(in)
begin
if(in[7:8]== 2'b00)
begin if(in[9]==0)
y=2'b00;
else if (in[10:16]==7'b0000000)
y=2'b00;
else
y=2'b01; end
else if(in[7:8]== 2'b01)
begin if(in[9:12]==4'b0010 )
y=2'b01;
else if (in[9:10]==2'b0 & in[11]==1'b0)
y=2'b01;
else if (in[9:10]==2'b0 & in[11:12]==2'b11 & in[13:16]==4'b0)
y=2'b01;

```

```

        else y= 2'b10;    end

else if(in[7:8]== 2'b10)
    begin if(in[9]==0)
        y=2'b10;
        else if (in[9:16]==8'b10000000)
            y=2'b10;
        else
            y=2'b11;    end
    else
        y=2'b11;

```

```

end

```

```

endmodule

```

```

module mux_c(c1,c2,c3,c4,sel,y

```

```

);
input [1:16] c1,c2,c3,c4;
input [1:0]sel;
output reg [1:16]y;
always@(*)
begin
    if(sel==2'b00)
        y=c1;
    if(sel==2'b01)
        y=c2;
        if(sel==2'b10)
            y=c3;
        if(sel==2'b11)
            y=c4;

```

end

endmodule

module modified_input_block(in,y1,y2,y3,y4

);
input [1:16] in;
output [1:16] y1,y2,y3,y4;

assign y1=in>>2;
assign y2=in>>3;
assign y3=in>>5;
assign y4=in>>6;

endmodule

module adder(a,b,ci,sum,cout);

input a,b,ci;
output sum,cout;
assign sum=a^b^ci;
assign cout= (a&b)|ci&(a|b);

endmodule

module adder_ful(in1,in2,sum ,cout

```

);
input [1:16] in1, in2;
output [1:16] sum;
output cout;
wire [14:0] c;

adder f0(in1[16], in2[16], 0, sum[16], c[0]);
adder f1(in1[15], in2[15], c[0], sum[15], c[1]);
adder f2(in1[14], in2[14], c[1], sum[14], c[2]);
adder f3(in1[13], in2[13], c[2], sum[13], c[3]);
adder f4(in1[12], in2[12], c[3], sum[12], c[4]);
adder f5(in1[11], in2[11], c[4], sum[11], c[5]);
adder f6(in1[10], in2[10], c[5], sum[10], c[6]);
adder f7(in1[9], in2[9], c[6], sum[9], c[7]);
adder f8(in1[8], in2[8], c[7], sum[8], c[8]);
adder f9(in1[7], in2[7], c[8], sum[7], c[9]);
adder f10(in1[6], in2[6], c[9], sum[6], c[10]);
adder f11(in1[5], in2[5], c[10], sum[5], c[11]);
adder f12(in1[4], in2[4], c[11], sum[4], c[12]);
adder f13(in1[3], in2[3], c[12], sum[3], c[13]);
adder f14(in1[2], in2[2], c[13], sum[2], c[14]);
adder f15(in1[1], in2[1], c[14], sum[1], cout);

```

```

endmodule

module mux_2to1(a, b, sel, y)

);
input [0:7] a, b;
input sel;

```

```

output [0:7] y;
assign y= sel?a:b;
endmodule
module selective_1_comp( Q,sel,q1,q2,q3,q4,q5,q6,q7,q8

);
input [9:16] Q;
input sel;
output q1,q2,q3,q4,q5,q6,q7,q8;

assign q1=Q[9];
assign q2=Q[10]^sel;
assign q3=Q[11]^sel;
assign q4=Q[12]^sel;
assign q5=Q[13]^sel;
assign q6=Q[14]^sel;
assign q7=Q[15]^sel;
assign q8=Q[16]^sel;

```

```

endmodule

```

```

module aafu(In,Y

```

```

);
input [0:16] In;
output [1:8] Y;
wire [1:8] y;

wire [1:16] modIn;
wire [1:0] A;
wire [1:16] t1,t2,t3,t4,t5,Q,sum;
wire p,cout;
wire [1:8] r;

```

```

SIGN_conv S(In[1:9],In[0],In[10:16],modIn);
DOMAIN_Decoder D(modIn[7:16],A);
mux_c

```

```
M1(16'b00000000001000000,16'b00000000001010000,16'b00000000001101100,16'b00000000001110100,A,t1);
```

```
    assign p=modIn[1]|modIn[2] | modIn[3]|modIn[4]|modIn[5]| modIn[6]|(modIn[7]&modIn[8]);  
    modified_input_block B(modIn[1:16],t2,t3,t4,t5);  
    mux_c M2(t2,t3,t4,t5,A,Q);  
    adder_ful F(t1,Q,sum,cout);
```

```
    mux_2to1 M3(8'b10000000,sum[9:16],p,r);
```

```
    selective_1_comp C(r,In[0],y[1],y[2],y[3],y[4],y[5],y[6],y[7],y[8]);  
    assign Y= In[0]?8'b00000001+y:y;
```

```
endmodule
```

Test Bench

```
`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 01.09.2025 10:17:31
// Design Name:
// Module Name: tb_aafu
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
/////////////////////////////////////////////////////////////////
```

```
module tb_aafu(
);
    reg [0:16] in;
    wire [1:8] y;

    aafu uut(in,y);
    initial
    begin
```

```
in=17'b11111110010000000;#10;$display("in=%b,y=%b",in,y);//-7
in=17'b11111110011000000;#10;$display("in=%b,y=%b",in,y);//-6.5
in=17'b11111110100000000;#10;$display("in=%b,y=%b",in,y);//-6
```

```

in=17'b11111110101000000;#10;$display("in=%b,y=%b",in,y);/-5.5
in=17'b11111110110000000;#10;$display("in=%b,y=%b",in,y);/-5
in=17'b11111110111000000;#10;$display("in=%b,y=%b",in,y);/-4.5
in=17'b1111111000000000;#10;$display("in=%b,y=%b",in,y);/-4
in=17'b1111111001000000;#10;$display("in=%b,y=%b",in,y);/-3.5
in=17'b1111111010000000;#10;$display("in=%b,y=%b",in,y);/-3
in=17'b1111111011000000;#10;$display("in=%b,y=%b",in,y);/-2.5
in=17'b1111111100000000;#10;$display("in=%b,y=%b",in,y);/-2
in=17'b1111111101000000;#10;$display("in=%b,y=%b",in,y);/-1.5
in=17'b1111111110000000;#10;$display("in=%b,y=%b",in,y); /-1
in=17'b1111111111000000;#10;$display("in=%b,y=%b",in,y); /-0.5
in=17'b00000000001000000;#10;$display("in=%b,y=%b",in,y);/// 0.5,
in=17'b00000000010000000;#10;$display("in=%b,y=%b",in,y);///1,
in=17'b00000000011000000;#10;$display("in=%b,y=%b",in,y);///1.5,
in=17'b00000000100000000;#10;$display("in=%b,y=%b",in,y);///2,
in=17'b00000000101000000;#10;$display("in=%b,y=%b",in,y);///2.5,
in=17'b00000000110000000;#10;$display("in=%b,y=%b",in,y);///3,
in=17'b00000000111000000;#10;$display("in=%b,y=%b",in,y);///3.5,
in=17'b000000001000000000;#10;$display("in=%b,y=%b",in,y);///4,
in=17'b000000001001000000;#10;$display("in=%b,y=%b",in,y);///4.5,
in=17'b000000001010000000;#10;$display("in=%b,y=%b",in,y);///5,
in=17'b000000001011000000;#10;$display("in=%b,y=%b",in,y);///5.5
in=17'b000000001100000000;#10;$display("in=%b,y=%b",in,y);///6,
in=17'b000000001101000000;#10;$display("in=%b,y=%b",in,y);///6.6
in=17'b000000001110000000;#10;$display("in=%b,y=%b",in,y);///7,

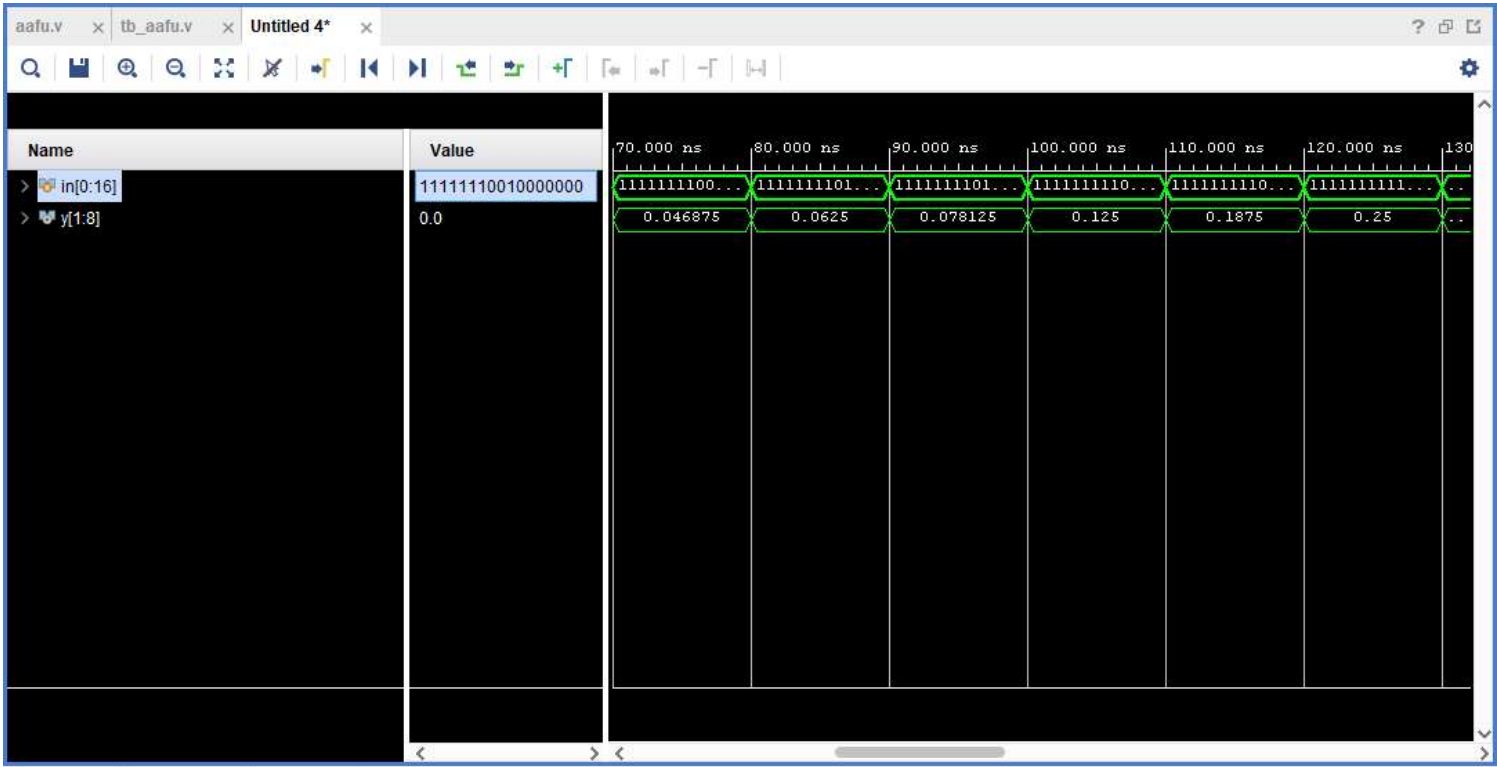
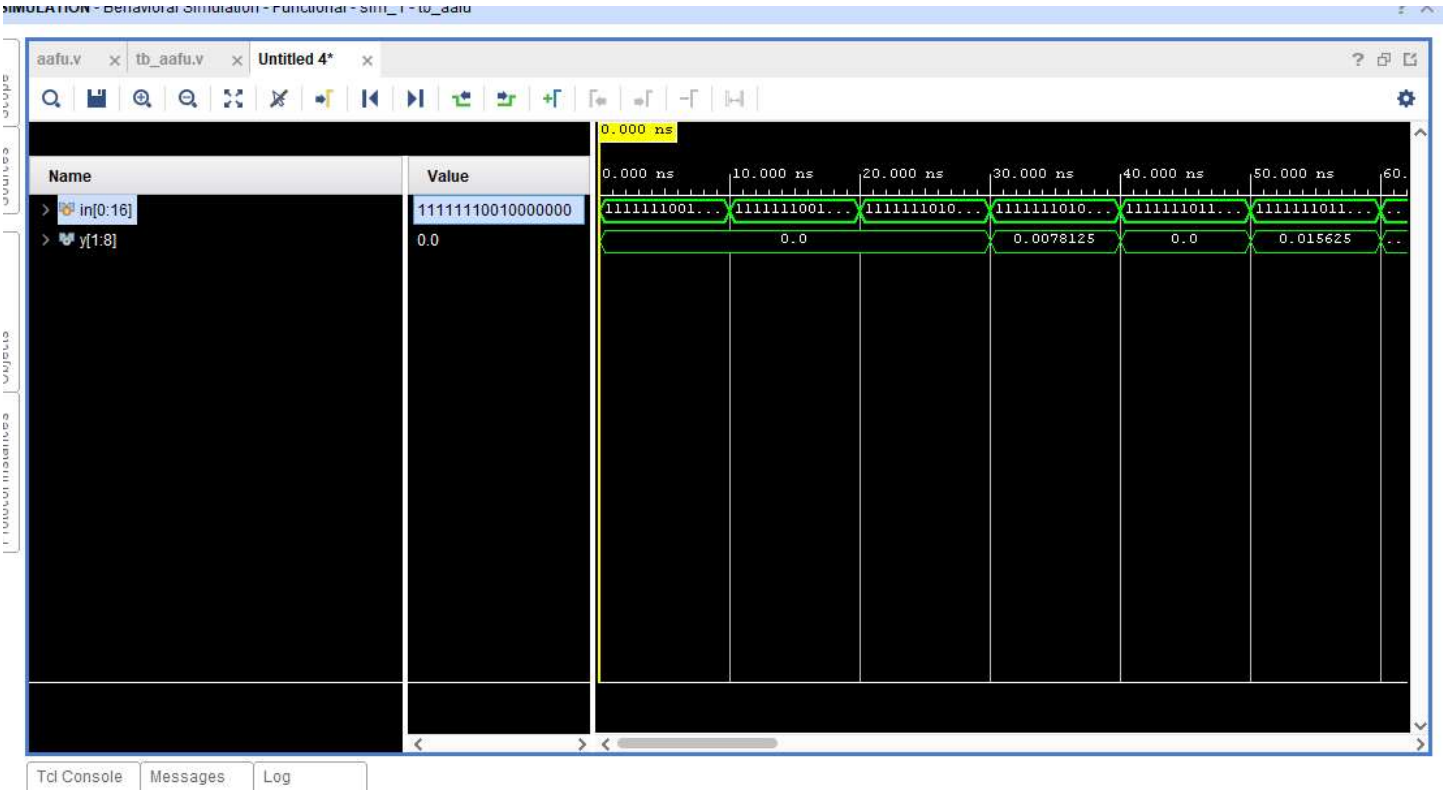
```

\$finish;

end

endmodule

Simulation Waveform



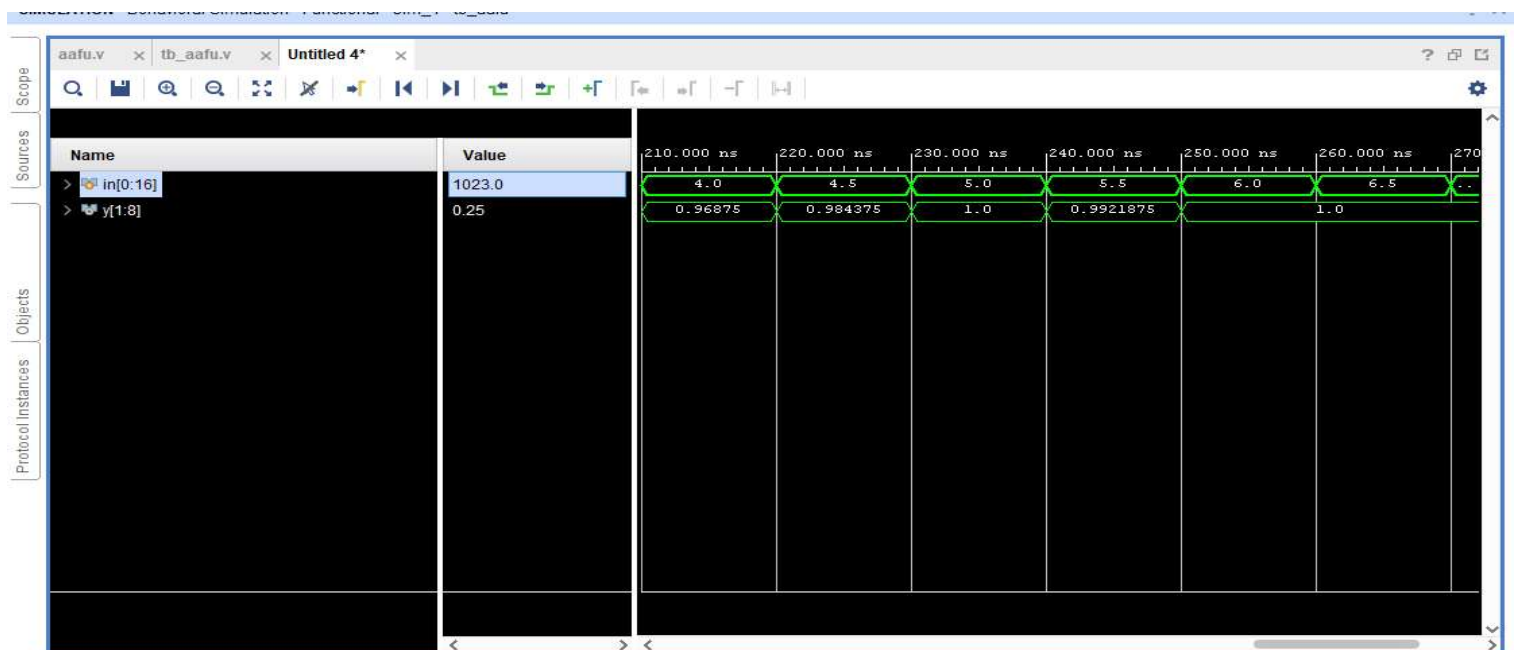
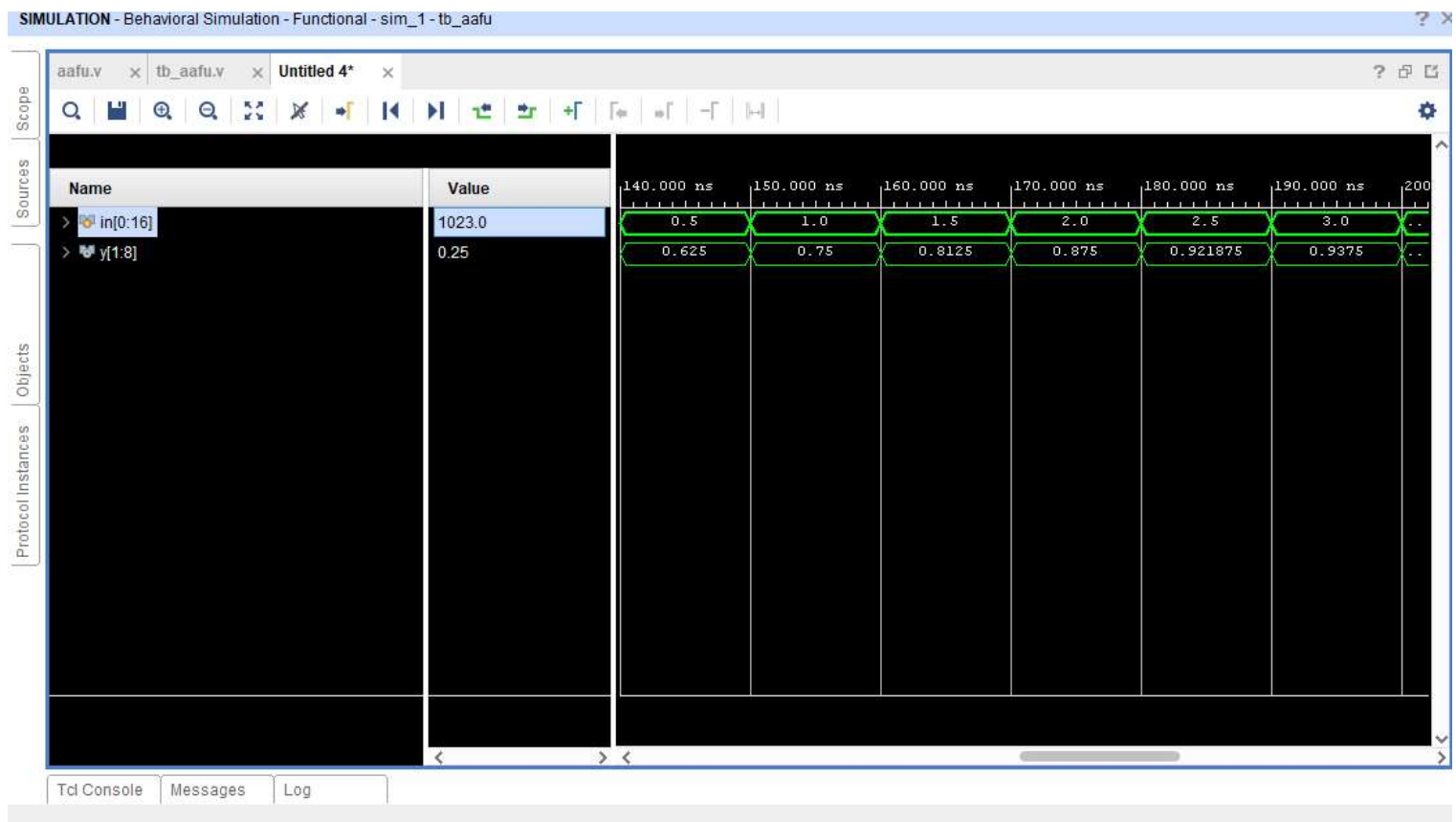
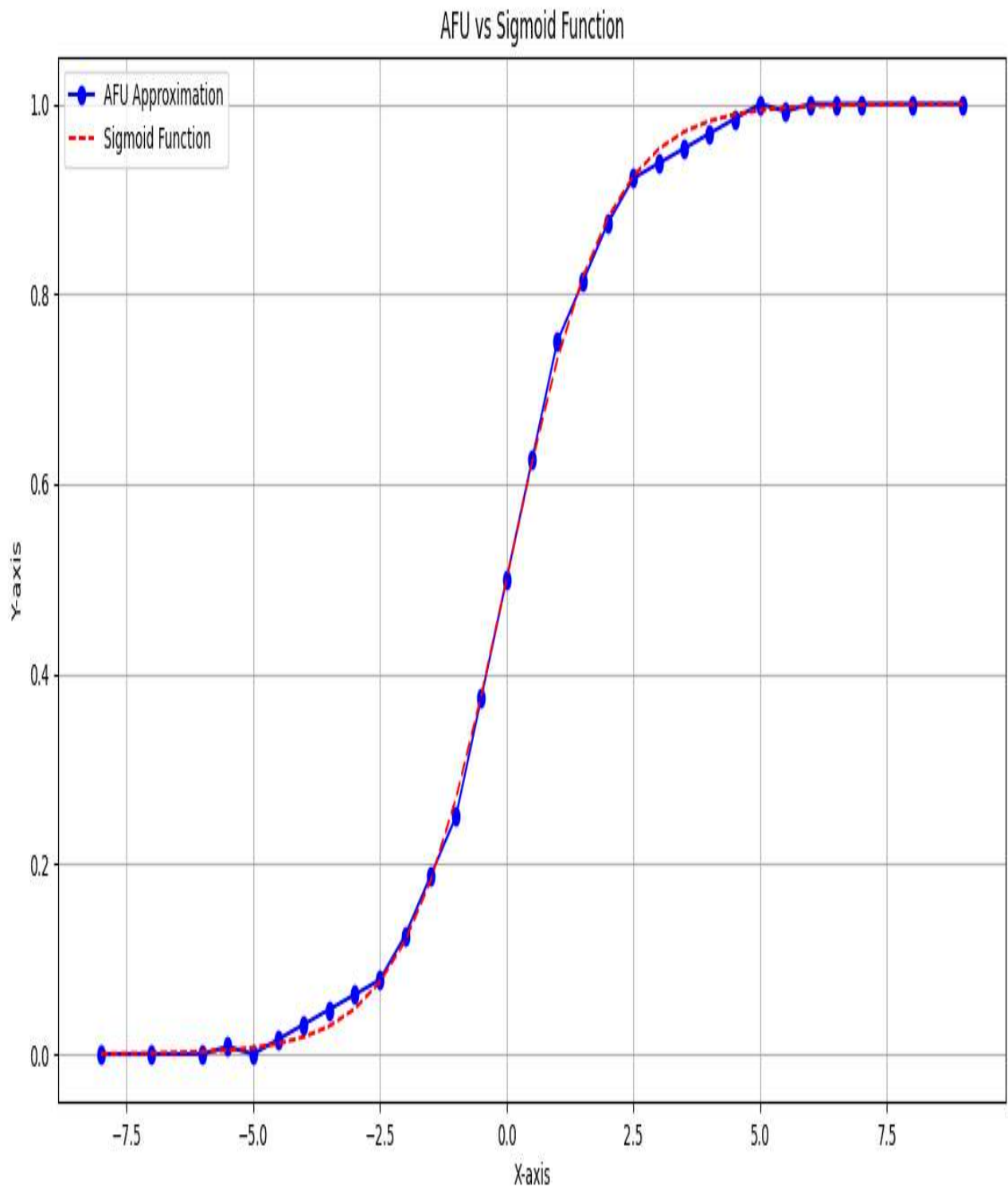


Fig : AFU I/O simulation result

Graph and ObservationTable

Figure 1

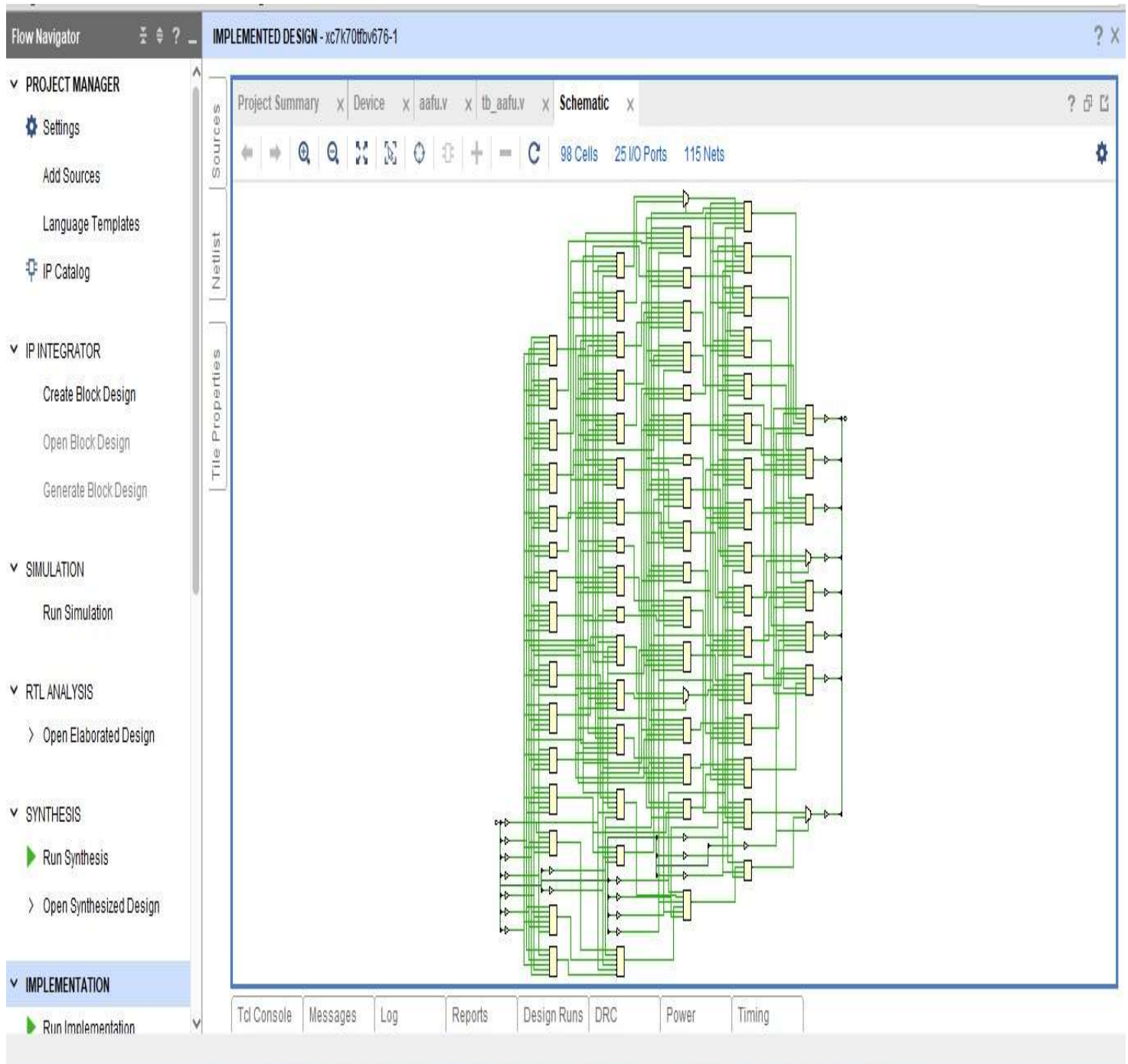
— □ X



X	AFU	Sigmoid	Error (AFU-Sig)

-8.00	0.00000	0.00034	-0.00034
-7.00	0.00000	0.00091	-0.00091
-6.00	0.00000	0.00247	-0.00247
-5.50	0.00781	0.00407	0.00374
-5.00	0.00000	0.00669	-0.00669
-4.50	0.01562	0.01099	0.00464
-4.00	0.03125	0.01799	0.01326
-3.50	0.04688	0.02931	0.01756
-3.00	0.06250	0.04743	0.01507
-2.50	0.07812	0.07586	0.00227
-2.00	0.12500	0.11920	0.00580
-1.50	0.18750	0.18243	0.00507
-1.00	0.25000	0.26894	-0.01894
-0.50	0.37500	0.37754	-0.00254
0.00	0.50000	0.50000	0.00000
0.50	0.62500	0.62246	0.00254
1.00	0.75000	0.73106	0.01894
1.50	0.81250	0.81757	-0.00507
2.00	0.87500	0.88080	-0.00580
2.50	0.92185	0.92414	-0.00229
3.00	0.93750	0.95257	-0.01507
3.50	0.95310	0.97069	-0.01759
4.00	0.96875	0.98201	-0.01326
4.50	0.98437	0.98901	-0.00464
5.00	1.00000	0.99331	0.00669
5.50	0.99218	0.99593	-0.00375
6.00	1.00000	0.99753	0.00247
6.50	1.00000	0.99850	0.00150
7.00	1.00000	0.99909	0.00091
8.00	1.00000	0.99966	0.00034
9.00	1.00000	0.99988	0.00012

RTL Schematic



Application ,Future Scope & Conclusion

Applications

The proposed sigmoid activation function unit (AFU) can be used in various hardware-based neural network applications where fast and efficient nonlinear computation is required. It is suitable for implementation in multilayer perceptron (MLP) networks for classification, regression, and pattern recognition tasks. The AFU can also be integrated into hardware accelerators for machine learning, signal processing systems, image processing, and real-time decision-making applications. Additionally, this design can be employed in FPGA- and ASIC-based AI systems where low power consumption and reduced latency are critical.

Future Scope

The current AFU design can be further enhanced by extending support to multiple activation functions such as ReLU, tanh, and leaky ReLU within the same architecture. Precision can be improved by adopting higher bit-width or adaptive fixed-point representations. The architecture may also be optimized using multiplier-less techniques such as distributed arithmetic to further reduce area and power consumption. In future work, the proposed AFU can be integrated with complete MLP or deep neural network hardware architectures and deployed in real-time applications such as autonomous systems, biomedical signal analysis, and edge AI devices.

Conclusion

In this project, a sigmoid activation function unit (AFU) has been successfully designed and implemented using Verilog and simulated using the Xilinx Vivado tool. The proposed design demonstrates an efficient hardware realization of a nonlinear activation function, which is a key component in neural network inference models. RTL synthesis and timing simulation results confirm the correctness and feasibility of the implementation.

The developed AFU provides a foundation for building hardware-based neural network architectures with improved performance and reduced computational complexity. By implementing the activation function at the hardware level, the design enables faster processing and is suitable for real-time and resource-constrained applications. Overall, this work validates the effectiveness of Verilog-based AFU design and highlights its potential for integration into advanced machine learning and signal processing systems.

1. [Efficient Multiplier-less Perceptron Architecture for Realization of Multilayer Perceptron Inference Models | Circuits, Systems, and Signal Processing](#)
2. <https://nptel.ac.in/courses/106105165>
3. [http://ebook.pldworld.com/_eBook/FPGA%EF%BC%8FHDL/-Eng-/Verilog%20digital%20systems%20design%20\(Navabi\).pdf](http://ebook.pldworld.com/_eBook/FPGA%EF%BC%8FHDL/-Eng-/Verilog%20digital%20systems%20design%20(Navabi).pdf)